CI/CD & DevSecOps Pipeline Implementation

Author: Nirmalya Das
Course: PW Skills DevOps Assignment
Topic: Continuous Integration, Testing, Security, and Continuous Deployment

---

1. Introduction

Continuous Integration and Continuous Deployment (CI/CD) are important DevOps practices that automate the process of building, testing, and deploying applications. CI/CD pipelines improve software quality, reduce manual errors, and accelerate development cycles.

In this assignment, a complete **CI/CD and DevSecOps pipeline** was implemented using GitHub Actions and various DevOps tools. The pipeline automatically builds the application, runs multiple testing stages, scans for security vulnerabilities, and deploys the application to a staging environment.

---

2. Tools and Technologies Used

Category	Tools
Version Control	GitHub
CI/CD Automation	GitHub Actions
Build Automation	Jenkins
Programming Language	Python
Web Framework	Flask
Containerization	Docker
Performance Testing	k6
Security Testing	OWASP ZAP

---

3. Pipeline Overview

The implemented CI/CD pipeline performs the following automated stages:

Stage	Description
1	Checkout the repository
2	Setup Python environment
3	Install project dependencies
4	Run unit tests
5	Run integration tests
6	Execute performance tests
7	Perform security vulnerability scan
8	Build Docker image
9	Deploy application to staging environment

Task 1: Continuous Integration (CI)

Objective

To set up a CI pipeline that automatically builds the application whenever new code is pushed to the repository.

Tools Used

- GitHub
- GitHub Actions
- Python

Steps Performed

1. Created a GitHub repository for the project.
2. Developed a simple Python application.
3. Configured a CI pipeline using GitHub Actions.
4. The pipeline triggers automatically whenever code is pushed to the **main branch**.

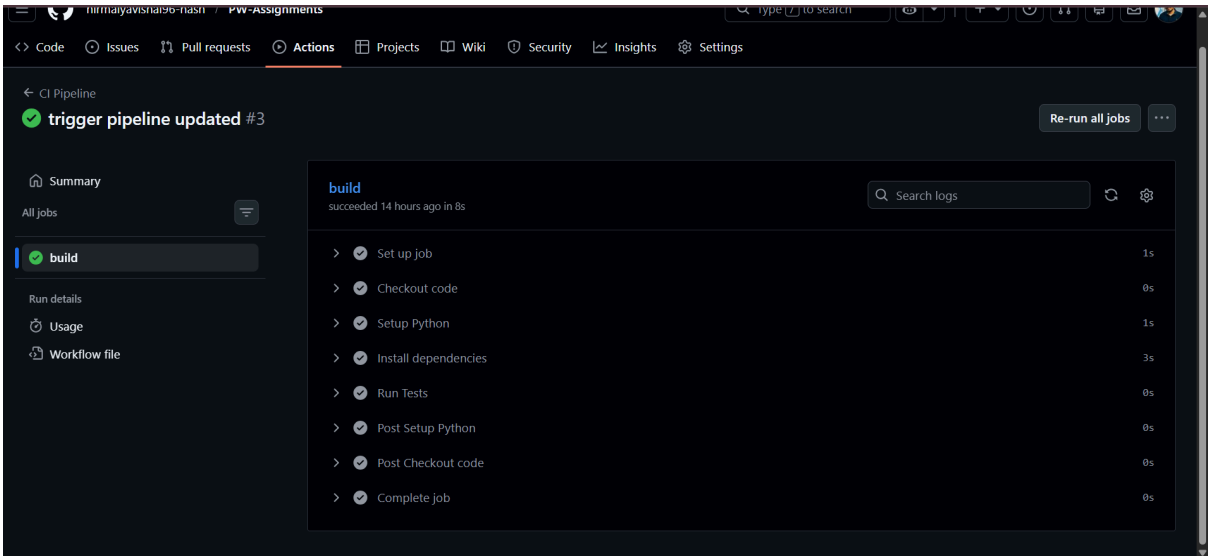
Workflow Configuration

Workflow file location:

.github/workflows/ci.yml

The pipeline automatically installs dependencies and executes the testing stages.

Screenshot



Task 2: Jenkins Automated Build

Objective

To configure Jenkins to automatically build the project by pulling code from the GitHub repository.

Tools Used

- Jenkins
- GitHub
- Python

Steps Performed

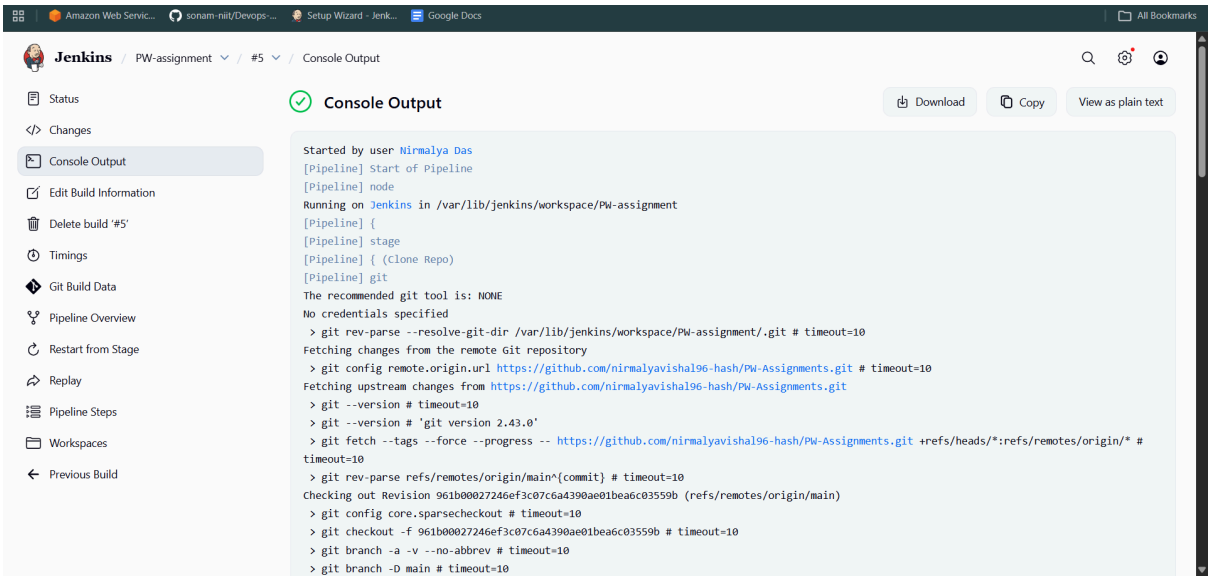
1. Installed Jenkins on the system.

2. Created a Jenkins pipeline job.
3. Connected Jenkins with the GitHub repository.
4. Configured Jenkins to clone the repository and execute the build pipeline.

Example Jenkins Pipeline Stage

```
stage('Clone Repo') {
   steps {
      git 'https://github.com/repository-url'
   }
}
```

Screenshot



Task 3: Unit Testing

Objective

To verify individual components of the application using automated unit tests.

Tool Used

pytest

Steps Performed

1. Created Python functions in the application.
2. Implemented unit tests using pytest.
3. Integrated the unit tests into the CI pipeline.
4. The pipeline automatically runs tests whenever code is pushed.

Example Command

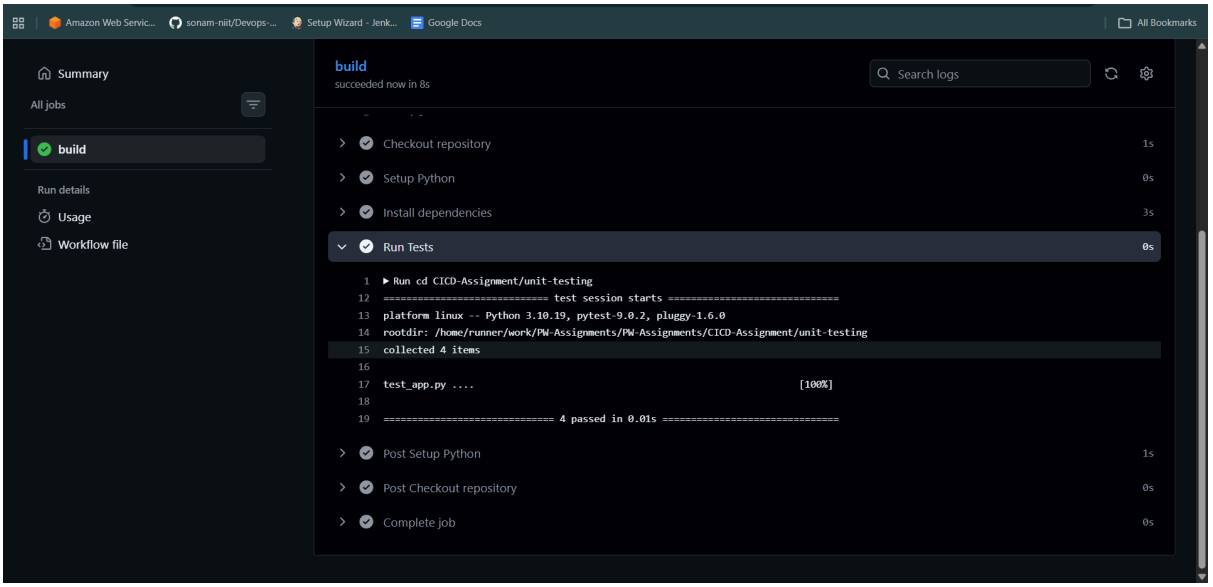
pytest

Example Output

4 passed in 0.02s

Unit testing ensures that each function of the application works correctly.

Screenshot



Task 4: Integration Testing

Objective

To test communication between different application components, particularly API endpoints.

Tools Used

- Flask
- pytest
- requests library

Steps Performed

1. Created a REST API using Flask.
2. Implemented endpoints for the application.
3. Wrote integration tests to verify API responses.
4. Integrated integration tests into the CI pipeline.

Example API Endpoint

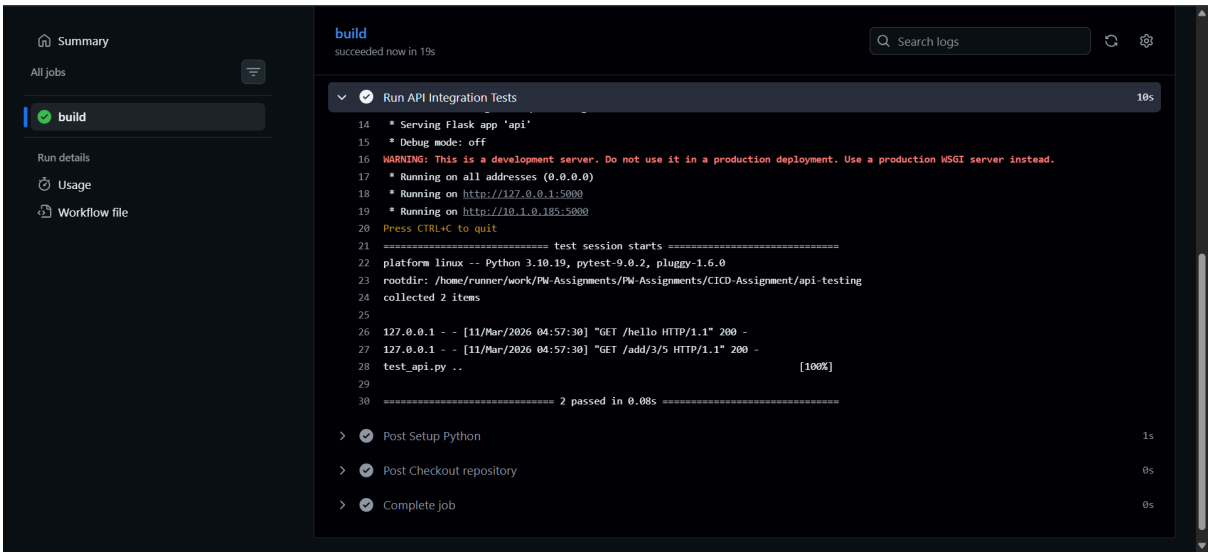
http://127.0.0.1:5000/hello

Example Response

{"message": "Hello DevOps"}

Integration testing verifies that different modules of the application interact correctly.

Screenshot



Task 5: Performance Testing

Objective

To measure application performance using load testing.

Tool Used

k6

Steps Performed

1. Created a performance testing script.
2. Simulated multiple user requests.
3. Measured response time and request handling capacity.
4. Integrated performance testing into the CI pipeline.

Example Command

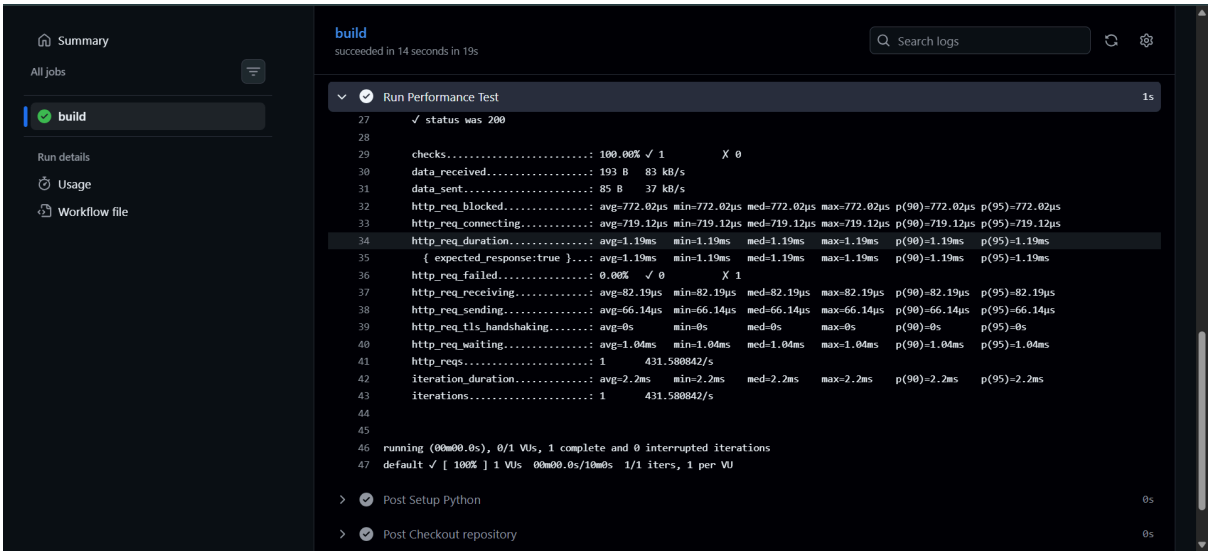
k6 run loadtest.js

Example Metrics

http_req_duration: avg=20ms
checks: 100% passed

This testing ensures that the application can handle concurrent user requests efficiently.

Screenshot



Task 6: Security Testing

Objective

To identify potential security vulnerabilities in the application using automated scanning tools.

Tool Used

OWASP ZAP

Steps Performed

1. Integrated OWASP ZAP into the CI pipeline.
2. Executed automated vulnerability scanning.
3. Checked for common security issues including:

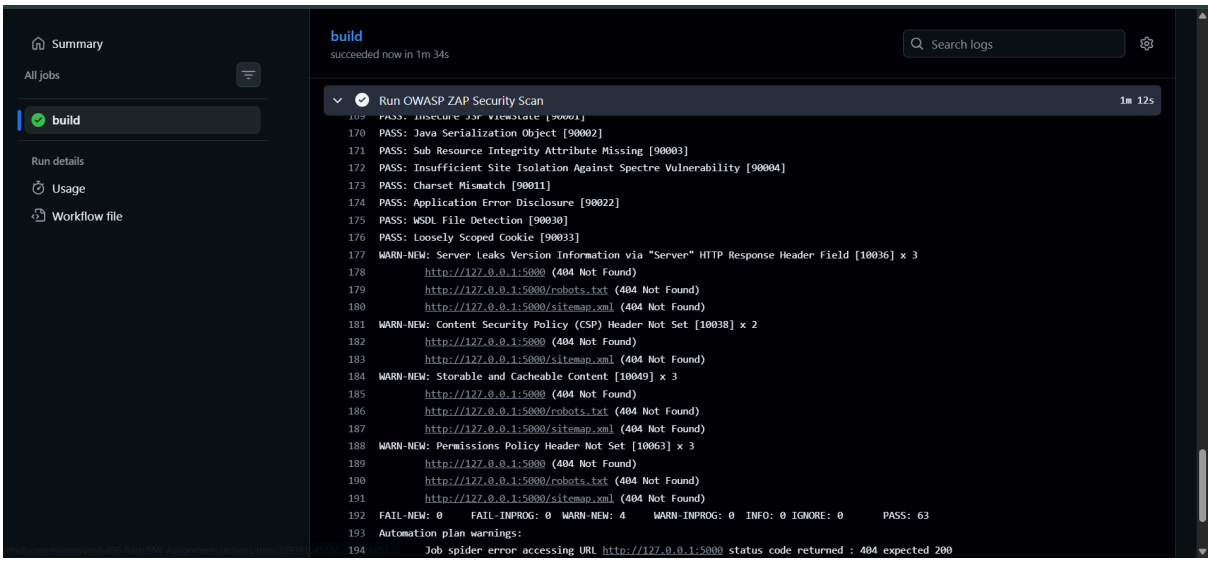
- Missing security headers
- Information disclosure
- Cross-site scripting risks
- insecure configurations

Example Scan Command

docker run --rm ghcr.io/zaproxy/zaproxy:stable zap-baseline.py -t http://127.0.0.1:5000

Security testing helps ensure the application is protected against common vulnerabilities.

Screenshot



Task 7: Continuous Deployment (CD)

Objective

To automatically deploy the application to a staging environment after successful testing.

Tools Used

- Docker
- GitHub Actions

Steps Performed

1. Created a Dockerfile for the application.
2. Built a Docker image in the CI pipeline.
3. Deployed the container to a staging environment.

Example Docker Commands

Build Docker Image

docker build -t devops-api .

Run Container

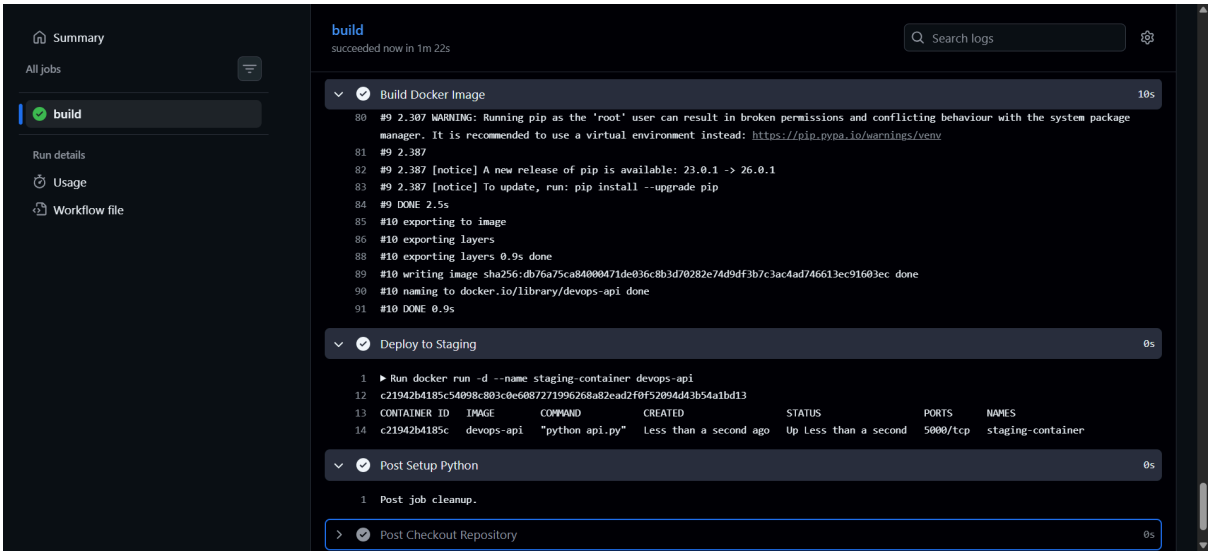
docker run -d devops-api

Verify Deployment

docker ps

The application is deployed as a Docker container representing the staging environment.

Screenshot



CI/CD Pipeline Execution Flow

Code Push
↓
GitHub Actions Pipeline Triggered
↓
Install Dependencies
↓
Run Unit Tests

↓
Run Integration Tests
↓
Run Performance Tests
↓
Run Security Scan
↓
Build Docker Image
↓
Deploy to Staging Environment

Benefits of CI/CD Pipeline

Feature	Benefit
Automated Testing	Detects bugs early
Security Scanning	Identifies vulnerabilities
Performance Testing	Ensures application stability
Continuous Deployment	Faster delivery of updates
Containerization	Environment consistency

Conclusion

This project demonstrates the implementation of a **complete DevSecOps pipeline** integrating:

- Continuous Integration
- Automated Testing
- Performance Testing
- Security Scanning
- Continuous Deployment

The automated pipeline ensures **efficient, reliable, and secure software delivery**, reflecting real-world DevOps practices.